

MICROSOFT FABRIC / DATA PLATFORMS

The challenges when implementing Microsoft Fabric — and the hidden cost of compute.

Fabric's promise is seductive: one SaaS platform, one copy of data in OneLake, one bill. But the unit on that bill — the Capacity Unit — behaves like nothing your finance team has priced before. Every notebook, pipeline, refresh and query draws from the same shared pool, and the true cost of a workload only becomes visible after it has already throttled everyone else.

FIVE CHALLENGES THAT SURFACE AFTER GO-LIVE

01

One capacity, every workload

Pipelines, Spark jobs, warehouses and Power BI all consume the same Capacity Units. A single runaway notebook can throttle the CFO's dashboards — and nobody owns the collision.

02

Smoothing hides the true bill

Bursting and 24-hour smoothing defer consumption rather than cancel it. Teams discover on Monday that Friday's backfill has mortgaged the capacity — and interactive users pay the throttling penalty.

03

Sizing is a guess, upgrades are a doubling

F-SKUs scale in powers of two. When an F64 runs hot, the next step is an F128 at twice the cost — a blunt instrument for what is usually one inefficient pipeline.

04

Full reloads burn the pool

Hand-built pipelines default to reprocessing whole tables. In a capacity model, every unnecessary full load is compute you have already bought being spent to produce nothing new.

05

Governance lags the workspaces

Fabric makes it effortless to create workspaces, lakehouses and shortcuts — far faster than data owners can define contracts, quality rules and lineage. The result is OneLake filling with ungoverned copies whose ingestion cost recurs every day, but whose fitness for use was never agreed.

The way out: spend compute only on governed change

In a capacity-priced platform, the cheapest record is the one you never reprocess. The organisations that keep Fabric affordable ingest incrementally — streaming only what changed — and validate every record against a machine-readable data contract (ODCS) before it lands, so bad data never consumes downstream Capacity Units at all. Schema, quality thresholds, ownership and lineage are enforced at the point of entry, with a human in the loop for exceptions. Compute stops being an open tap and becomes an auditable line item: every Capacity Unit spent maps to a governed, incremental change.

Stop paying to reprocess data you already trust.

QBricks ingests from system of record to governed lakehouse — streaming, incremental, ODCS-native, running natively on Microsoft Fabric.

[Request a demo →](#)